

第9次作业总结

P578 题5

题目

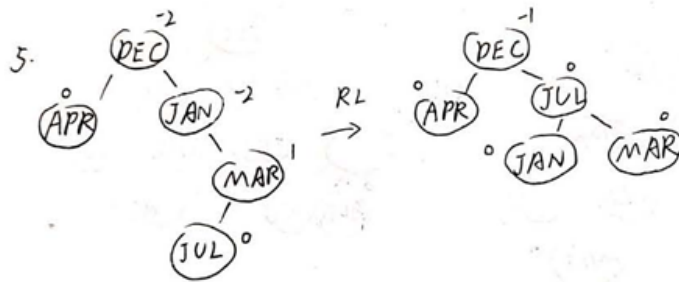
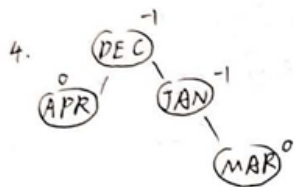
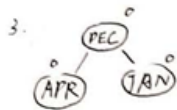
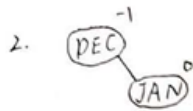
Start with an empty AVL tree and perform the following sequence of insertions: DECEMBER, JANUARY, APRIL, MARCH, JULY, AUGUST, OCTOBER, FEBRUARY, NOVEMBER, MAY, JUNE. Use the strategy of *Avl :: Insert* to perform each insert. Draw the AVL tree following each insertion and state the rotation type (if any) for each insert.

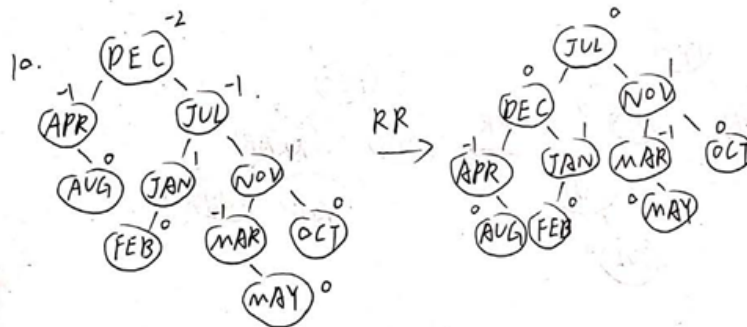
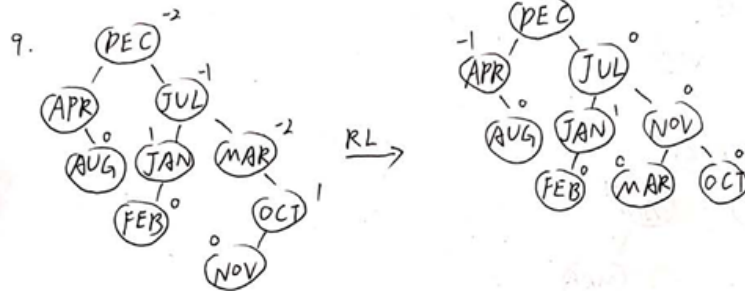
参考答案

可参考如下解答：

P578

5.





P578 题9

题目

Write an algorithm to delete the element with key k from an AVL tree. The resulting tree should be restructured if necessary. Show that the time required for this is $O(\log n)$ when there are n nodes in the tree. [Hint: If k is not in a leaf, then replace k by the largest value in its left subtree or the smallest value in its right subtree. Continue until the deletion propagates to a leaf. Deletion from a leaf can be handled using the reverse of the transformations used for insertion.]

参考答案

解题思路：

- 总体思路：
 1. 查找需要删除的 k 点：从根开始进行比较，每次比较过后决定向左还是向右
 2. 执行删除操作：这里可以分为三种情况
 1. k 在叶子结点，则直接删除该叶子，父结点的某个指针置为空
 2. k 只有一个子结点，则用它唯一的子结点进行替代
 3. k 有两个子结点，则不能直接进行删除，否则查找树的结构将被破坏。因此需要用该结点的中序前驱（左子树中的最大值，即树中恰好比 k 结点小的值）或中序后继（右子树中的最小值，即树中恰好比 k 结点大的值）进行替换。同时， k 结点的前驱或后继结点一定最多只有一个子结点（左子树中的最大值一定没有右子树，同理右子树中的最小值一定没有左子树）。

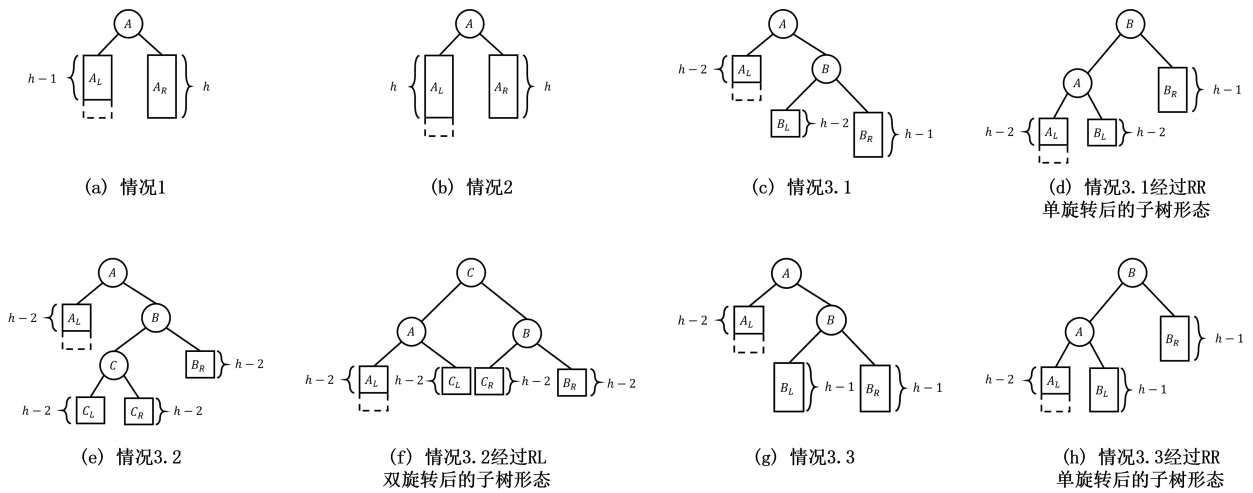
综合三种情况，最终删除一定会“传递”到叶子或单子结点情况。

3. 向上回溯恢复AVL平衡：进行了结点的删除，高度减少，且该子树高度也可能减少，因此需要沿着删除路径一路向上，更新结点的高度并进行平衡修复。

前面1、2步较为简单，这里我们主要关注第3步。

- 删除后的操作：

当在结点的某棵子树上删除一个结点，导致该子树的高度减小1时，可能会出现图所示的5种情况。由于在左、右子树上删除结点的情况是完全轴对称的，此处假设在结点A的左子树上删除一个结点。



- 情况1: 图(a)中，原来的AVL树是完全平衡二叉树，左、右子树高度都为 h ，删除一个结点后，左子树高度减1，整棵树仍然保持平衡，以A为根结点的子树高度保持不变。
- 情况2: 图(b)中，原来的AVL树的结点A的平衡度是+1，删除左子树的结点后，结点A的平衡度变成0，仍然符合AVL树的定义。但是以结点为根结点的子树高度也相应减小了1，因此要继续回溯，检查A的父结点的平衡性。
- 图(c)、图(e)、图(g)中，原来结点A的平衡度为-1，删除结点后A失去平衡，因此需要调整树的平衡。A的右子树形态对应以下3种情况。

- 情况3.1: 图(c)对应RR情况，因此进行RR单旋转，旋转后的子树形态如图(d)所示，这会导致以A为根结点的子树（现在根结点变成了B）高度-1，即子树原本高度为 h 而现在为 $h-1$ ，因此需要回溯检查当前子树的父结点的平衡性。
- 情况3.2: 图(e)对应RL情况(注意， C_L 和 C_R 两棵子树有可能有一棵高度只有 $h-3$ ，依然符合AVL树的定义，这里处理广义的情况)，进行RL双旋转后的子树形态如图(f)所示。RL双旋转后，以A为根结点的子树高度减小，需要检查A的祖先结点的平衡性。
- 情况3.3: 图(g)中，结点B仍然平衡，可以对结点A执行RR单旋转或RL双旋转，一般选择便于执行的RR单旋转。执行RR单旋转后的子树形态如图(h)所示，执行RR单旋转后，以A为根结点的子树(现在高度不变)不再需要检查A结点的祖先结点的平衡性。

• 四种失衡情况与旋转（LL、RR、LR、RL）：

- 命名规则：第一个字母是失衡结点（即 $|\text{平衡度}| = 2$ 的结点）的“高子树方向”，第二个字母是高子树中“更高的子树方向”
- 具体旋转：
 - LL型：即左边过高，且“高在左边的左边”，因此需要进行一次右旋



- RR型：即右边过高，且“高在右边的右边”，因此需要进行一次左旋



- LR型：结点 z 失衡，左子树过高： $BF(z) = +2$ ，但左孩子 y 的右子树更高： $BF(y) = -1$ ，因此需要先对 y 左旋然后对 z 右旋



- LR型：结点 z 失衡，右子树过高： $BF(z) = -2$ ，但右孩子 y 的左子树更高： $BF(y) = +1$ ，因此需要先对 y 右旋然后对 z 左旋



- 时间复杂度：因为每次删除结点时都是从根结点开始一直遍历到叶子结点，所以每次都要进行 h 次遍历（ h 为树高），在二叉树中 $h = \log(n)$ ，因此，时间复杂度为 $O(\log(n))$ 。

总结以上：AVL 树删除首先按照二叉搜索树规则查找并删除关键字为 k 的结点。若 k 有两个子结点，则用其中序前驱或后继替换，并将删除操作递归地传递到叶子结点。删除后，沿删除路径向上更新结点高度并检查平衡因子。若某结点失衡，则通过单旋转或双旋转恢复平衡。由于 AVL 树高度为 $O(\log n)$ ，查找、删除以及平衡调整均在 $O(\log n)$ 时间内完成，因此整个删除操作的时间复杂度为 $O(\log n)$ 。